

PlayList Manager

Project 03 - Graduation Project

Presented by :
Nourhan Ahmed Kamal

Project Overview

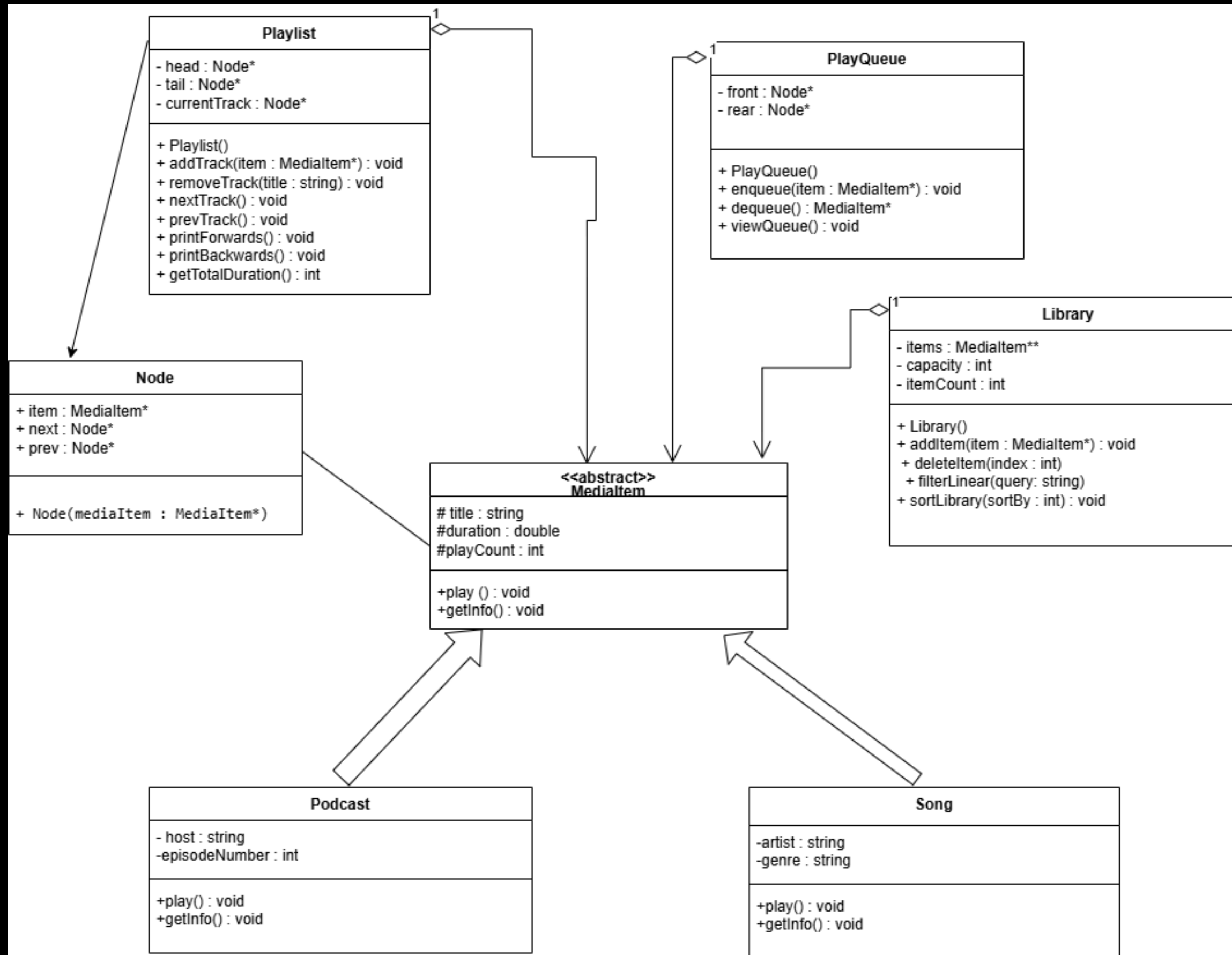
Core Idea:

A C++ console-based music application that manages a library of songs and podcasts, custom dynamic playlists, and playback queues.

Key Features:

- Dynamic Playlist: Smooth forward and backward track navigation using a custom Doubly Linked List.
- Up Next Queue: Sequential track queueing built from scratch.
- Algorithms: Exact binary search, custom sorting with comparison counting, and recursive operations.

<https://github.com/gogonona384-crypto/PlayList-Manager>



UML-class-diagram



OOP(ARCHITECTURE)

Inheritance: Song inherits properties (title, duration) from MediaItem.

Polymorphism: Song overrides play() to stream audio logic, while Podcast overrides it for episode details.

Polymorphism Implementation: Abstract MediaItem class with virtual void play() = 0;.

Inheritance: Derived classes (Song and Podcast) override play() to provide custom streaming logic.

```
C MediaItem.h | README.md | .gitignore | .gitattributes | main.cpp |M, M | Song.h | Podcast.h | C N | ...
include > C Song.h
1  #ifndef SONG_H
2  #define SONG_H
3
4  #include "MediaItem.h"
5
6  class Song : public MediaItem
7  {
8  private:
9      string artist;
10     string genre;
11
12 public:
13     Song(string t = "", string a = "", int d = 0, string g = "")
14         : MediaItem(t, d), artist(a), genre(g) {}
15
16     void play() override
17     {
18         incrementPlayCount();
19         cout << "[SONG] streaming audio..." << endl;
20     }
21
22     void getInfo() const override
23     {
24         cout << "[SONG] " << title << " | Artist: " << artist
25             << " | Duration: " << duration << "s | Genre: " << genre
26             << " | Plays: " << playCount << endl;
27     }
28
29     string getArtist() const
```

```
C MediaItem.h | README.md | .gitignore | .gitattributes | main.cpp | Song.h | Podcast.h | Node.h | ...
include > C MediaItem.h
1  #ifndef MEDIAITEM_H
2  #define MEDIAITEM_H
3
4  #include <string>
5  #include <iostream>
6  using namespace std;
7
8  class MediaItem {
9  protected:
10     string title;
11     int duration;
12     int playCount;
13
14 public:
15     MediaItem(string t = "", int d = 0) : title(t), duration(d), playCount(0) {}
16     virtual ~MediaItem() = default;
17
18     virtual void play() = 0;
19     virtual void getInfo() const = 0;
20
21     string getTitle() const { return title; }
22     int getDuration() const { return duration; }
23     int getPlayCount() const { return playCount; }
24     void incrementPlayCount() { playCount++; }
25
26     friend ostream& operator<<(ostream& os, const MediaItem& item) {
27         os << "Title: " << item.title << ", Duration: " << item.duration << " seconds, Play Count: " << item.playCount;
28         return os;
29     }
```


Data structure & Algorithms Complexity

Doubly Linked List (Playlist): Built from scratch to enable $O(1)$ bidirectional (next / prev) traversal.
Queue (PlayQueue): Custom FIFO structure from scratch for managing the playback queue.
Recursion: Used for calculating totalDuration and printBackwards

Operation	Structure	Time Complexity
Search by Title	Sorted Array	$O(\log n)$ (Binary Search)
Sort Library	Array	$O(n^2)$ (Selection Sort)
Enqueue / Dequeue	Custom Queue	$O(1)$
Traverse Tracks	Doubly Linked List	$O(1)$ per step

Live Demo & Conclusion

Interactive Walkthrough:

Library Management: Add & view songs/podcasts.

Playlist Traversal: Demonstrate next and previous navigation on Doubly Linked List.

Up Next Queue: Add and play tracks sequentially from custom Queue.

Search & Sort: Execute Binary Search and Selection Sort with comparison metrics.

Stats & Recursion: Show total library duration computed recursively.

ThankYou